

TECH

STATE MANAGEMENT

STATE MANAGEMENT

STATE MANAGEMENT

35 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК [react.dev](#) / [redux](#) / [zustand](#) / [mobx](#) / [xstate](#) / [tanstack](#)

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 ЛЁГКИЙ	LOCAL T Q1 · Q2 · Q3 · Q4 · Q5	CONTEXT	ОПТИМИЗАЦИЯ CONTEX	База: где жить состоянию и почему Context — не state-manager.
ДЕНЬ 2 СРЕДНИЙ	ВЫБОР STATE Q6 · Q7 · Q8 · Q9 · Q10	REDUX vs	ZUSTAND vs	MOBX vs X Когда какой инструмент брать — частый вопрос на собеседе.
ДЕНЬ 3 ТЯЖЁЛЫЙ	SERVER STATE Q11 · Q12 · Q13	REACT QUERY		Главное разделение — клиентский vs серверный state.
ДЕНЬ 4 СРЕДНИЙ	BOUNDARIES TABS Q14 · Q15 · Q16 · Q17 · Q18	URL /	STORAGE /	MEMORY Где должен жить тот или иной кусок состояния.
ДЕНЬ 5 ТЯЖЁЛЫЙ	OPTIMISTIC ALIDATION Q19 · Q20 · Q21 · Q22	ROLLBACK	STALE	INV Тонкости работы с серверными данными и кэшем.
ДЕНЬ 6 ТЯЖЁЛЫЙ	NORMALIZATION ASYNC FSM Q23 · Q24 · Q25 · Q26 · Q27 · Q28	DERIVED	DUPLICATED	Архитектура хранилища: нормализация и производные данные.
ДЕНЬ 7 СРЕДНИЙ	PERMISSIONS ERF TESTS Q29 · Q30 · Q31 · Q32 · Q33 · Q34 · Q35	FLAGS MIGRATION	REAL-TIME	P Закрепление и операционные вопросы хранилища.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА ЛЁГКИЙ 5 ВОПРОСОВ

LOCAL CONTEXT ОПТИМИЗАЦИЯ CONTEXT**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

База: где жить состоянию и почему Context – не state-manager.

ВОПРОСЫ ДНЯ

- Q01 Когда достаточно local state?
- Q02 Когда нужен Context?
- Q03 Почему Context не всегда state management?
- Q04 Какие проблемы производительности может вызвать Context?
- Q05 Как оптимизировать Context?

Когда достаточно local state?

ОПИСАНИЕ

Local state (useState/useReducer внутри одного компонента) достаточен, пока данные не нужны соседним поддеревьям. Это самый дешёвый, простой и быстрый способ хранить состояние: открытие модальной, hover, локальная форма, тогглы. Когда данные нужны нескольким независимым компонентам – пора подумать о подъёме state, Context или внешнем сторе.

КАК РАБОТАЕТ

- 01 useState – для одного-двух простых значений.
- 02 useReducer – для нескольких связанных полей или сложного перехода.
- 03 Состояние живёт в инстансе компонента, исчезает при unmount.
- 04 Расшаренное состояние делается через подъём (lifting state up) к ближайшему общему предку.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 UI-флаги (открыто/закрыто, активная вкладка).
- 02 Поля формы внутри одного компонента.
- 03 Состояние анимации, хувер, ввод.
- 04 Кэш одного запроса локально внутри страницы.

ПОДВОДНЫЕ КАМНИ

- 01 Подъём state на 5 уровней вверх – болезненно. Лучше Context или стор.
- 02 Хранение тяжёлых вычислений в state без useMemo.
- 03 Локальный state, который нужно сохранить между монтировками – теряется. Нужен внешний слой.

КОД

// useState простой

```
const [open, setOpen] = useState(false);
```

// useReducer для связанных полей

```
const [s, d] = useReducer(reducer, { name: '', age: 0 });
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда useState заменить на useReducer?
- 02 Когда подъём state становится избыточным?
- 03 Можно ли local state синхронизировать между табами?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/managing-state>

Когда нужен Context?

ОПИСАНИЕ

Context нужен, когда данные нужно передать в несколько поддеревьев без prop-drilling, и эти данные меняются редко: тема, локаль, токен, текущий пользователь, конфиг. Если данные меняются часто – Context может вызвать массовые ререндеры (см. Q4).

КАК РАБОТАЕТ

- 01 `<Context.Provider value={...}>...</Context.Provider>`.
- 02 `useContext(Ctx)` подписывает компонент на изменения `value`.
- 03 При смене `value` (не `Object.is` равно) ререндерятся все `consumer`'ы.
- 04 React. мемо не защищает от Context-ререндера.
- 05 Альтернативы: подъём `state`, `render props`, внешний `store`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тема (`light/dark`).
- 02 Локаль / `i18n`-словари.
- 03 Текущий пользователь / `auth-state`.
- 04 Реестр маршрутов / роутер.
- 05 Динамические конфиги, фичи.

ПОДВОДНЫЕ КАМНИ

- 01 Context для часто меняющегося `state` – ререндер всех `consumer`'ов.
- 02 Один Context на всё приложение → любое изменение бьёт по дереву.
- 03 `Inline value={{...}}` → новый объект на каждый рендер → ложные ререндеры.

КОД

// Базовое

```
const ThemeCtx = React.createContext('light');
<ThemeCtx.Provider value={theme}>
  <App />
</ThemeCtx.Provider>;
const t = useContext(ThemeCtx);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем Context отличается от `store`?
- 02 Что произойдёт при `inline value`?
- 03 Как разбить большой Context?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/createContext>

Почему Context не всегда state management?

ОПИСАНИЕ

Context — механизм ПРОБРОСА значения, а не управления состоянием. Он не оптимизирует ререндеры (любое изменение value → все consumer'ы), не имеет селекторов, middleware, subscriptions. Для большого изменяющегося state нужен настоящий store (Redux, Zustand, MobX) — обычно через useSyncExternalStore под капотом.

КАК РАБОТАЕТ

- 01 Context = pub-sub без granular-подписок.
- 02 Любой setState в Provider → все consumer'ы перерендерятся.
- 03 Нет встроенных селекторов и мемо-сравнения.
- 04 Часто комбинируют: Context передаёт ССЫЛКУ на store, сам store — внешний.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание помогает не использовать Context там, где он не подходит.
- 02 Архитектурные обсуждения — где границы.
- 03 При выборе state-library.

ПОДВОДНЫЕ КАМНИ

- 01 Большой Context-state с частыми изменениями = performance-кошмар.
- 02 Использование Context как «бесплатного store» в обмен на лагающий UI.
- 03 Сложно дебажить — нет history/devtools.

КОД

// Антипаттерн — Context как глобальный state

```
const AppCtx = createContext({});  
// state в Provider — каждый setState ре-рендерит ВСЕ consumer'ы
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда Context достаточен, а когда нужен store?
- 02 Что меняет use-context-selector?
- 03 Чем Context отличается от Zustand?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useContext>

ВОПРОС Q04

Какие проблемы производительности может вызвать Context?

ОПИСАНИЕ

Главная — массовый ре-рендер: при изменении value все consumer'ы перерендериваются, даже те, кто читает только часть value. React.memo не помогает — ре-рендер из Context приходит сверху. Inline value={{...}} = новая ссылка каждый рендер = бесполезные обновления. Большой Context-state с частыми сменами = тормоза.

КАК РАБОТАЕТ

- 01 useContext(C) подписывает компонент на C.
- 02 При смене value по Object.is — все подписчики перерендериваются.
- 03 Селекторов из коробки нет: нельзя «реагировать только на одно поле».
- 04 Inline value создаёт новую ссылку каждый рендер.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Всегда стоит думать перед добавлением changing-state в Context.
- 02 Чтобы дебажить «почему всё ререндерится» — посмотреть Context-вершины.

ПОДВОДНЫЕ КАМНИ

- 01 Один Context = одно значение для всего поддерева.
- 02 В Provider value = inline-объект — все consumer'ы ре-рендерятся каждый раз.
- 03 Большой объект с разными полями — нет способа выбрать только нужное без external libs.

КОД

// Антипаттерн — inline value

```
<Ctx.Provider value={{ a, b }}>...</Ctx.Provider>  
// каждый рендер — новый объект, все consumer'ы ререндерятся
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем use-context-selector помогает?
- 02 Почему React.memo не спасает от Context-рендера?
- 03 Что делать, если Context действительно нужен?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useContext#optimizing-re-renders-when-passing-objects-and-functions>

Как оптимизировать Context?

ОПИСАНИЕ

Стратегии: разделить один большой Context на несколько узких (тема отдельно, юзер отдельно, конфиг отдельно). Стабилизировать value через useМемо. Разделить state и actions на два разных Context (читатели actions не ре-рендерятся при смене state). Использовать внешнюю библиотеку: use-context-selector, Zustand с Context-обёрткой.

КАК РАБОТАЕТ

- 01 Разбиение по типу данных: ThemeCtx, UserCtx, RouterCtx.
- 02 useМемо для value: `const value = useМемо(() => ({a, b}), [a, b])`.
- 03 Split state/actions: `{ state, dispatch }` → два Context.
- 04 use-context-selector: подписка только на нужный slice.
- 05 Zustand с createContext-обёрткой – хороший компромисс.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Когда Context уже есть и нужно выжать перфоманс.
- 02 При проектировании auth-state, где state меняется, а actions – нет.
- 03 В крупных приложениях с многими провайдерами.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком много Context-провайдеров → "provider hell".
- 02 useМемо с неправильными deps → всё равно новая ссылка.
- 03 Split state/actions нужно поддерживать вручную – сложно.

КОД

// Split state/actions

```
const StateCtx = createContext(null);
const DispatchCtx = createContext(null);
function Provider({ children }) {
  const [state, dispatch] = useReducer(r, init);
  return (
    <StateCtx.Provider value={state}>
      <DispatchCtx.Provider value={dispatch}>
        {children}
      </DispatchCtx.Provider>
    </StateCtx.Provider>
  );
}
```

// dispatch стабилен, его читатели не ре-рендерятся

МОГУТ ДОСПРОСИТЬ

- 01 Почему dispatch из useReducer стабилен?
- 02 Чем use-context-selector отличается от useContext?
- 03 Когда Zustand лучше Context?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useContext#optimizing-re-renders-when-passing-objects-and-functions>

ВЫБОР REDUX vs ZUSTAND vs MOBX vs XSTATE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Когда какой инструмент брать – частый вопрос на собесед.

ВОПРОСЫ ДНЯ

- Q06 Когда нужен Redux?
- Q07 Когда Redux будет overkill?
- Q08 Когда выбрать Zustand?
- Q09 Когда выбрать MobX?
- Q10 Когда выбрать XState?

ВОПРОС Q06

Когда нужен Redux?

ОПИСАНИЕ

Redux уместен, когда нужно сложное предсказуемое управление client-state с прозрачной историей: time-travel, dev-tools, undo/redo, плагины middleware (saga, observable). Подходит для крупных приложений с многими разработчиками: жёсткие правила (action → reducer → store) сильно структурируют код. С Redux Toolkit (RTK) boilerplate сильно меньше – это современный стандарт.

КАК РАБОТАЕТ

- 01 Один store, иммутабельный update через reducer.
- 02 Action – описание изменения; reducer – pure function.
- 03 Middleware – побочные эффекты (thunk, saga, observable).
- 04 RTK = createSlice / createAsyncThunk / RTK-Query.
- 05 Подписка через useSelector – селекторы с мемо.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие SPA с множеством разработчиков.
- 02 Нужны DevTools time-travel, replay.
- 03 Сложная бизнес-логика, которую важно отделить от UI.
- 04 Командные требования к структуре (enterprise).

ПОДВОДНЫЕ КАМНИ

- 01 Old-school Redux – много boilerplate. RTK решает.
- 02 Серверный state в Redux часто избыточен – лучше react-query.
- 03 Селекторы без мемо – лишние ре-рендеры.
- 04 Глобальный state с частыми изменениями требует оптимизаций (reselect).

КОД

// RTK slice

```
const slice = createSlice({
  name: 'counter',
  initialState: { v: 0 },
  reducers: {
    inc: (s) => { s.v++ },           // immer внутри
    add: (s, a: PayloadAction<number>) => { s.v += a.payload },
  },
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем RTK отличается от классического Redux?
- 02 Когда Redux overkill?
- 03 Что делает reselect?

ПОЛНАЯ СТАТЬЯ

<https://redux-toolkit.js.org/>

ВОПРОС Q07

Когда Redux будет overkill?

ОПИСАНИЕ

Маленькие и средние приложения, где state помещается в useState/useReducer + react-query. Если глобальный client-state почти отсутствует, а большая часть данных – серверная, Redux не даёт ценности, только усложняет код.

КАК РАБОТАЕТ

- 01 Маленький проект – местный state + контекст для темы/юзера.
- 02 react-query / SWR закрывает серверный state.
- 03 Глобальный client-state на 1-2 значения – Zustand (минимальный boilerplate).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Стартапы / MVP – react-query + Zustand хватает.
- 02 Сайты с малым state – useState + Context.
- 03 Микро-фронт-енды – каждый со своим лёгким store.

ПОДВОДНЫЕ КАМНИ

- 01 Принимать Redux "на всякий случай" – лишний код и кривая обучения для команды.
- 02 Использовать Redux для server-state и обнаруживать, что он плохо подходит.
- 03 Без RTK классический Redux отпугивает новичков.

КОД

// Минимально – Zustand

```
import { create } from 'zustand';
const useStore = create<{ count: number; inc: () => void }>((set) => ({
  count: 0,
  inc: () => set((s) => ({ count: s.count + 1 })),
}));
```

МОГУТ ДОСПРОСИТЬ

- 01 Что выбрать для маленького проекта?
- 02 Когда нужно мигрировать на Redux?
- 03 Чем Zustand проще Redux?

ПОЛНАЯ СТАТЬЯ

<https://docs.pmnd.rs/zustand/getting-started/introduction>

Когда выбрать Zustand?

ОПИСАНИЕ

Zustand – лёгкая state-библиотека (~3 KB). Подходит, когда нужен глобальный store без церемоний: один `create()` с `initial state` и `actions`, использование через хук с селектором. Идеален для middle-проектов и для виджетов / UI-state, которые не вписываются в `react-query`.

КАК РАБОТАЕТ

- 01 `create<State>()((set, get) => ({...}))` – определение store.
- 02 `useStore(selector)` – компонент подписывается на нужный slice.
- 03 Селектор – функция `(state) => slice`.
- 04 `set` частично сливает state, мутации не нужны.
- 05 middleware: `persist`, `devtools`, `immer`.
- 06 Поддержка SSR, `useSyncExternalStore` под капотом.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mid-проекты с глобальным client-state.
- 02 Theme/auth/notifications/global-modal-state.
- 03 Виджеты с переиспользуемой логикой (drag-drop, modal).
- 04 Когда Redux ощущается тяжёлым.

ПОДВОДНЫЕ КАМНИ

- 01 Без shallow-сравнения селектор объектом → лишние re-рендеры.
- 02 Глобальный store, в котором что попало – превращается в хаос. Нужны границы.
- 03 `Persist/devtools middleware` – стоят bundle-размером.

КОД

// Store + selector

```
const useCart = create<CartStore>((set) => ({
  items: [],
  add: (i) => set((s) => ({ items: [...s.items, i] })),
}));
const items = useCart((s) => s.items);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем Zustand отличается от Redux?
 - 02 Что такое shallow-сравнение и зачем оно?
 - 03 Какие middleware есть в Zustand?
-

ПОЛНАЯ СТАТЬЯ

<https://docs.pmnd.rs/zustand/getting-started/introduction>

Когда выбрать MobX?

ОПИСАНИЕ

MobX – реактивный state с прокси-наблюдением. Подходит командам, привыкшим к OOP/reactive-style: классы-store с observable полями, derived (computed), actions. Хорош для сложных domain-models, где много derived данных, и где нужно тонкое автоматическое отслеживание зависимостей.

КАК РАБОТАЕТ

- 01 makeAutoObservable / makeObservable превращает поля в observable.
- 02 computed – кэшированные derived-значения.
- 03 action – мутация observable (под strict-mode обязательно).
- 04 observer-обёртка React-компонента подписывается на наблюдаемые.
- 05 Только то, что прочитано в render, отслеживается.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Сложные domain-модели (CRM, медицинские системы).
- 02 Большие SPA с derived-вычислениями.
- 03 Команды с опытом MobX или Vue (схожая идея).
- 04 Когда хочется писать классы-state и не следить за иммутабельностью.

ПОДВОДНЫЕ КАМНИ

- 01 Magic – иногда трудно понять, почему ререндерится.
- 02 DevTools слабее, чем у Redux.
- 03 Time-travel и snapshot – сложнее.
- 04 Mutability противоречит React-привычкам.

КОД

// Базовый класс-store

```
class CartStore {
  items: Item[] = [];
  constructor() { makeAutoObservable(this); }
  add(i: Item) { this.items.push(i); }
  get total() { return this.items.reduce((a, x) => a + x.price, 0); }
}
const Cart = observer(({ store }: { store: CartStore }) => (
  <div>{store.total}</div>
));
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем computed отличается от обычного метода?
- 02 Что такое observer и зачем?
- 03 Можно ли использовать MobX без классов?

ПОЛНАЯ СТАТЬЯ

<https://mobx.js.org/README.html>

Когда выбрать XState?

ОПИСАНИЕ

XState — библиотека `state-machines` и `statecharts`. Подходит, когда есть конечный набор состояний и сложные правила переходов: визарды, плееры, протоколы соединений, checkout. Преимущества: формальное описание, exhaustive-обработка событий, визуализация, невозможность невалидных переходов.

КАК РАБОТАЕТ

- 01 `createMachine({ id, initial, states, on })` — описание автомата.
- 02 Состояния и события явно перечислены.
- 03 `Guards / actions / services` — побочные эффекты.
- 04 `useMachine(machine)` — React-интеграция.
- 05 Statechart-визуализатор — рисует диаграмму.
- 06 Actor model в XState v5 — общение между машинами.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Сложные multi-step формы и wizards.
- 02 Видео/аудио-плееры.
- 03 WebSocket-протоколы с handshake.
- 04 Async-loading flows с `retry/cancel`.
- 05 Логика, где можно нарисовать диаграмму состояний.

ПОДВОДНЫЕ КАМНИ

- 01 Кривая обучения — нужно понимать statecharts.
- 02 Overkill для простых toggle'ов.
- 03 Bundle size заметный (XState 5 меньше).
- 04 Тестирование машин требует отдельных подходов.

КОД

// Минимальная машина

```
const machine = createMachine({
  id: 'fetch',
  initial: 'idle',
  states: {
    idle: { on: { FETCH: 'loading' } },
    loading: { on: { OK: 'success', FAIL: 'error' } },
    success: { type: 'final' },
    error: { on: { RETRY: 'loading' } },
  },
});
```

```
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем XState лучше useReducer?
 - 02 Что такое actor model?
 - 03 Как тестировать state machine?
-

ПОЛНАЯ СТАТЬЯ

<https://state.ly.ai/docs>

SERVER STATE**REACT QUERY****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Главное разделение – клиентский vs серверный state.

ВОПРОСЫ ДНЯ

Q11 Когда выбрать React Query / TanStack Query?

Q12 Чем server state отличается от client state?

Q13 Почему server state не стоит класть в Redux без необходимости?

ВОПРОС Q11

Когда выбрать React Query / TanStack Query?

ОПИСАНИЕ

React Query (TanStack Query) – специализированное решение для server-state: кэширование, дедупликация, refetch, stale-time, фоновые обновления, optimistic updates, infinite-query, pagination. Используется ВСЕГДА, когда приложение работает с REST/GraphQL – упрощает 90% связанной асинхронной логики.

КАК РАБОТАЕТ

- 01 useQuery({ queryKey, queryFn, staleTime, ... }) – загрузка с кэшем.
- 02 queryKey – массив, идентифицирующий запрос. Меняется → refetch.
- 03 useMutation – для записей с onMutate/onSuccess/onError.
- 04 queryClient – глобальный кэш + методы invalidate, setQueryData.
- 05 useSuspenseQuery – Suspense-режим.
- 06 Persist + offline – отдельные плагины.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые REST / GraphQL endpoints.
- 02 Замена ad-hoc useEffect+fetch.
- 03 Замена thunks-ов в Redux для async-данных.
- 04 Optimistic updates, infinite scroll.
- 05 Background refetch при focus/reconnect.

ПОДВОДНЫЕ КАМНИ

- 01 Не state-manager – для client-state нужен другой инструмент.
- 02 Без правильного queryKey – кэш не работает.
- 03 staleTime по умолчанию 0 → лишние запросы. Настраивать.
- 04 В RSC (Next App Router) роль смещается – данные грузятся на сервере.

КОД

```
// useQuery + useMutation
```

```
const { data, isPending } = useQuery({
  queryKey: ['user', id],
  queryFn: () => api.getUser(id),
  staleTime: 60_000,
});
const m = useMutation({
  mutationFn: api.updateUser,
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['user'] }),
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем queryKey важен для кэша?
- 02 Что такое staleTime vs cacheTime?
- 03 Как комбинировать с Zustand?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/overview>

ВОПРОС Q12

Чем server state отличается от client state?

ОПИСАНИЕ

Client state – то, что принадлежит клиенту: UI-флаги, локальные настройки, текущая страница. Server state – копия данных с сервера: пользователи, посты, заказы. Server state особенный: он async, может устаревать, может конфликтовать с другими клиентами, требует дедупликации, фонового обновления, инвалидации. Отсюда отдельные инструменты (react-query, SWR, Apollo).

КАК РАБОТАЕТ

- 01 Client state: фактически живёт в памяти браузера, меняется только локально.
- 02 Server state: реальный источник на сервере, локальная копия – кэш.
- 03 Server state нужен механизм синхронизации: refetch при focus, при reconnect, по интервалу, по mutation.
- 04 Server state может быть stale (устаревшим) – требуется понятие staleTime.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Везде, где есть API: разделять на server-state (через react-query) и client-state (через Zustand/Redux/local).
- 02 Это разделение упрощает архитектуру и снимает 80% проблем с async-логикой.

ПОДВОДНЫЕ КАМНИ

- 01 Класть server state в Redux – много дубликатов (loading/error/data повторяется в каждом slice).
- 02 Не понимать stale-time – лишние запросы или устаревшие данные.
- 03 Смешивать в одной структуре – теряется ясность.

КОД

// Разделение

```
// server-state
const { data: user } = useQuery({ queryKey: ['user'], queryFn: api.getUser });

// client-state
const isModalOpen = useStore((s) => s.modal.open);
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие данные считаются server state?
- 02 Почему server state нельзя «просто» хранить в Redux?
- 03 Что такое cache-and-network strategy?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/important-defaults>

ВОПРОС Q13

Почему server state не стоит класть в Redux без необходимости?

ОПИСАНИЕ

Redux рассчитан на client-state с предсказуемыми переходами. Server state – асинхронный, кэшируемый, фоновый, обновляемый – требует кучи дополнительной логики (loading/error/data, dedup, refetch, stale time). Тащить всё это в Redux вручную – дублирование того, что уже сделано в react-query/SWR. RTK Query – компромисс: это часть RTK, специально для server-state.

КАК РАБОТАЕТ

- 01 Каждый асинхронный запрос в Redux = свой slice с loading/error/data/payload.
- 02 Дедупликация запросов – руками.
- 03 Refetch при focus/reconnect – руками.
- 04 Stale-time – руками.
- 05 Optimistic updates с откатом – руками.
- 06 RTK Query закрывает большую часть, react-query – ещё больше.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Старые приложения, где Redux уже стоит – RTK Query встраивается.
- 02 Новые проекты – react-query / TanStack Query.
- 03 Критичные сценарии с предсказуемой сагой-логикой иногда оправдывают thunk-saga.

ПОДВОДНЫЕ КАМНИ

- 01 Дубль состояния (server в Redux + локальный кэш) – источник багов.
- 02 Селекторы по асинхронным данным становятся сложными.
- 03 Нет встроенной дедупликации – два компонента → два запроса.

КОД

// RTK Query

```
const api = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  endpoints: (b) => ({
    getUser: b.query<User, string>({ query: (id) => `/users/${id}` }),
  }),
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем RTK Query отличается от react-query?
- 02 Можно ли использовать оба в одном проекте?

03 Что такое thunks и зачем они были?

ПОЛНАЯ СТАТЬЯ

<https://redux-toolkit.js.org/rtk-query/overview>

ДЕНЬ 4

НАГРУЗКА

СРЕДНИЙ

5 ВОПРОСОВ

BOUNDARIES**URL / STORAGE / MEMORY****TABS****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Где должен жить тот или иной кусок состояния.

ВОПРОСЫ ДНЯ

- Q14 Как проектировать state boundaries?
- Q15 Что должно жить в URL?
- Q16 Что должно жить в localStorage/sessionStorage?
- Q17 Что должно жить в memory state?
- Q18 Как синхронизировать tabs?

Как проектировать state boundaries?

ОПИСАНИЕ

Разделение state по уровням и владельцам. Принципы: локальный state – внутри одного компонента; общий – у ближайшего общего предка или в URL/Storage; глобальный client – в store; server – в react-query кэше. Каждое значение должно иметь чёткий «дом» и понятный жизненный цикл.

КАК РАБОТАЕТ

- 01 Local: useState внутри компонента.
- 02 Lifted: useState в общем предке + проп вниз.
- 03 Cross-page: URL (фильтры, страница) или Storage (session/persist).
- 04 Global client: Zustand/Redux/Context.
- 05 Server: react-query.
- 06 Любой кусок state – кто-то один владеет, остальные читают.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Архитектурные ревью.
- 02 Onboarding нового кода.
- 03 Решение «куда положить новое значение».

ПОДВОДНЫЕ КАМНИ

- 01 Дубли – один и тот же data в local и global.
- 02 Слишком много global state – становится хаотично.
- 03 URL как мусорка для всего.

КОД

// Boundary-чек-лист

```
// нужно ли делиться с другими компонентами?  
// - нет → local  
// - да, в одном поддереве → lift up  
// - да, через всё → global store  
// нужно ли восстанавливать после reload?  
// - да, поделено URL → URL  
// - да, скрыто → storage  
// это копия серверных данных?  
// - да → react-query
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое "single source of truth"?

02 Когда state «уехал не туда»?

03 Как ловить дубли?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/managing-state>

Что должно жить в URL?

ОПИСАНИЕ

URL – лучшее место для state, который должен быть shareable (можно отправить ссылку), bookmarkable (сохранить), и survivable между перезагрузками. Кандидаты: текущая страница, фильтры списка, активная вкладка, параметры поиска, sort. Не клади в URL: временные UI-флаги, секреты, большие объекты.

КАК РАБОТАЕТ

- 01 Path: /users/:id – идентификатор сущности.
- 02 Query string: ?q=react&sort=date – фильтры/сортировка.
- 03 Hash: #section – анкер.
- 04 Router (React Router, Next.js) – API для чтения/записи.
- 05 URL – string, всегда требует парсинга/валидации.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Поиск, фильтры, пагинация.
- 02 Текущая открытая страница / вкладка.
- 03 State, которым делятся в чате/email.
- 04 Глубокие ссылки в приложение.

ПОДВОДНЫЕ КАМНИ

- 01 URL длиной в 2000+ символов – ломаются разные сервисы.
- 02 Секреты в URL – попадают в логи / referer.
- 03 Несинхронность URL и UI – багает back-button.
- 04 Большие объекты в URL – кодировать в base64 – анти-паттерн.

КОД

```
// Next.js App Router
```

```
const params = useSearchParams();
const q = params.get('q') ?? '';
router.push(`?q=${encodeURIComponent(newQ)}`);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое deep links?
- 02 Как защититься от XSS через URL?
- 03 Как валидировать query-параметры?

ПОЛНАЯ СТАТЬЯ

<https://nextjs.org/docs/app/api-reference/functions/use-search-params>

Что должно жить в localStorage/sessionStorage?

ОПИСАНИЕ

Storage — для state, который должен пережить перезагрузку вкладки. localStorage сохраняется до явного удаления; sessionStorage — до закрытия вкладки. Кандидаты: тема, локаль, drafts формы, недавние поиски, признаки auth (НЕ сам токен — лучше cookie httpOnly). Ограничения — 5-10 MB, синхронные операции, только string.

КАК РАБОТАЕТ

- 01 localStorage.setItem('k', JSON.stringify(v)).
- 02 Get: JSON.parse(localStorage.getItem('k') ?? 'null').
- 03 Storage event — синхронизация между вкладками (см. Q18).
- 04 Синхронные операции — большие объёмы блокируют main thread.
- 05 В private mode иногда выкидывает quota error.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тема (light/dark) — чтобы не моргала при reload.
- 02 Локаль / язык.
- 03 Drafts длинных форм.
- 04 Recent searches / истории.
- 05 UI-настройки (sidebar collapsed).

ПОДВОДНЫЕ КАМНИ

- 01 JWT в localStorage — XSS-уязвимость. Лучше httpOnly cookie.
- 02 Очень большие объекты — лагают reload.
- 03 Только string — JSON-сериализация теряет Date/Map/Set.
- 04 Quota — может выбросить exception.
- 05 В iframe с другим origin — недоступно.

КОД

// Wrapper с try/catch

```
function lsGet<T>(k: string, def: T): T {
  try {
    const r = localStorage.getItem(k);
    return r ? JSON.parse(r) : def;
  } catch { return def; }
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем sessionStorage отличается от localStorage?

02 Можно ли хранить токен в localStorage?

03 Что такое IndexedDB и когда он лучше?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

Что должно жить в memory state?

ОПИСАНИЕ

Memory state – то, что не нужно после reload и не критично для других вкладок. Это default для большинства состояний приложения: открытие модальных, hover, выбранная вкладка, формы (если не drafts), кэш данных. Memory быстрая, простая, не требует сериализации.

КАК РАБОТАЕТ

- 01 useState / useReducer – local memory.
- 02 Zustand / Redux – global memory.
- 03 Module-level переменные – простейший shared memory (но не реактивный).
- 04 react-query – server-cache в memory.
- 05 При reload – теряется (это by design).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 UI-состояния (модалки, тогглы).
- 02 Текущий выбор пользователя.
- 03 Кэш сервера (react-query).
- 04 Подсчёты, derived-вычисления.

ПОДВОДНЫЕ КАМНИ

- 01 Хранить в memory то, что должно переживать reload – теряется.
- 02 Хранить в memory секреты – сбросом перезагрузка дешёвая, но уязвимости XSS остаются.
- 03 Большие объекты в memory – рост heap.

КОД

```
// Часто это просто Zustand-store  
const useStore = create<MyState>(...)
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем memory state отличается от server cache?
- 02 Когда memory недостаточен?
- 03 Что произойдёт при reload?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/managing-state>

ВОПРОС Q18

Как синхронизировать tabs?

ОПИСАНИЕ

Между вкладками одного origin синхронизация делается через storage event (для localStorage), BroadcastChannel (современный API), SharedWorker (advanced).
Используется для синхронной выгрузки на logout, обновления auth-state, reflection «в другой вкладке корзина изменилась», live-syncing настроек.

КАК РАБОТАЕТ

- 01 window.addEventListener('storage', e) – событие при изменении localStorage в ДРУГОЙ вкладке.
- 02 BroadcastChannel('name').postMessage(...) – publisher/subscriber между вкладками.
- 03 SharedWorker – общий процесс между вкладками; сложнее.
- 04 react-query syncs across tabs (online + focus refetch).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Logout: одна вкладка вышла → остальные тоже.
- 02 Auth: refresh-токен обновлён в одной вкладке.
- 03 Настройки темы / языка.
- 04 Realtime-симуляция: уведомить другие вкладки об обновлении.

ПОДВОДНЫЕ КАМНИ

- 01 storage event срабатывает только в ДРУГИХ вкладках, не в вызывающей.
- 02 BroadcastChannel не во всех старых браузерах (IE/старая Safari).
- 03 Гонки: две вкладки одновременно меняют данные.
- 04 Безопасность: не транслировать секреты.

КОД

```
// BroadcastChannel
```

```
const ch = new BroadcastChannel('auth');
ch.postMessage({ type: 'logout' });
ch.onmessage = (e) => {
  if (e.data.type === 'logout') logoutLocally();
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем BroadcastChannel лучше storage event?
- 02 Что такое SharedWorker?
- 03 Как избежать гонок между вкладками?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/BroadcastChannel>

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 4 ВОПРОСА

OPTIMISTIC ROLLBACK STALE INVALIDATION**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Тонкости работы с серверными данными и кэшем.

ВОПРОСЫ ДНЯ

- Q19 Как работать с optimistic updates?
- Q20 Как откатывать optimistic update?
- Q21 Как обрабатывать stale data?
- Q22 Как инвалидировать cache?

Как работать с optimistic updates?

ОПИСАНИЕ

Optimistic update – UI обновляется СРАЗУ, не дожидаясь ответа сервера. Если сервер подтвердил – оставляем; если упал – откатываем. Делает UX отзывчивым: like, add to cart, чек-листы. Реализуется через mutation с onMutate (применить локально) → onError (откатить) → onSettled (рефетч/инвалидация).

КАК РАБОТАЕТ

- 01 react-query: useMutation({ onMutate, onError, onSettled }).
- 02 onMutate: получить cancelQueries, snapshot текущего кэша, применить optimistic.
- 03 onError: восстановить snapshot.
- 04 onSettled: invalidateQueries.
- 05 Альтернатива – собственная логика в Redux/Zustand с previousState.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Like / favorite кнопки.
- 02 Add to cart.
- 03 Toggle / checklist.
- 04 Inline-edit полей.
- 05 Ranking / drag-drop.

ПОДВОДНЫЕ КАМНИ

- 01 Без отката – UI обманывает пользователя при сбое.
- 02 Race с другими mutations – нужен cancelQueries.
- 03 Inconsistent state, если onError не сработал – пользователь видит «висящий» optimistic.
- 04 Сложно отладить, потому что состояние меняется быстро.

КОД

// react-query optimistic

```
useMutation({
  mutationFn: (next) => api.update(next),
  onMutate: async (next) => {
    await qc.cancelQueries({ queryKey: ['user'] });
    const prev = qc.getQueryData(['user']);
    qc.setQueryData(['user'], next);
    return { prev };
  },
  onError: (_e, _v, ctx) => {
```



```
    qc.setQueryData(['user'], ctx?.prev);  
  },  
  onSettled: () => qc.invalidateQueries({ queryKey: ['user'] } ),  
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Зачем cancelQueries в onMutate?
- 02 Что вернёт onMutate в onError?
- 03 Как делать optimistic для списка?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

Как откатывать optimistic update?

ОПИСАНИЕ

Перед применением optimistic нужно сохранить snapshot предыдущего состояния. При ошибке onError получает context от onMutate (где snapshot был сохранён) и восстанавливает кэш. Дополнительно – показать пользователю toast «не удалось», возможно автоматически повторить через retry.

КАК РАБОТАЕТ

- 01 Snapshot – getQueryData до изменения.
- 02 Возврат snapshot из onMutate как context.
- 03 В onError – setQueryData(snapshot).
- 04 Можно показать toast/alert с возможностью retry.
- 05 Финал – invalidateQueries для синхронизации.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой optimistic update должен иметь rollback.
- 02 Чем тяжелее ошибка – тем заметнее уведомление.
- 03 В UX-критичных операциях – диалог, что предложить retry.

ПОДВОДНЫЕ КАМНИ

- 01 Если в onMutate не сохранили snapshot – нечего откатывать.
- 02 Если несколько optimistic в очереди – порядок отката важен.
- 03 Иногда сервер не вернул ошибку, но изменения конфликтуют → нужно сравнивать версии.

КОД

// Полный паттерн

```
useMutation({
  mutationFn: api.update,
  onMutate: async (next) => {
    const prev = qc.getQueryData(KEY);
    qc.setQueryData(KEY, next);
    return { prev };
  },
  onError: (_e, _v, ctx) => {
    qc.setQueryData(KEY, ctx?.prev);
    toast.error('Не сохранилось, попробуйте снова');
  },
  onSettled: () => qc.invalidateQueries({ queryKey: KEY }),
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Что произойдёт, если `onError` упадёт?
- 02 Как откатывать несколько `mutations` подряд?
- 03 Чем `onSettled` отличается от `onSuccess`?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

Как обрабатывать stale data?

ОПИСАНИЕ

Stale = устаревшие данные относительно сервера. Стратегии: stale-while-revalidate (показать кэш + фоновое обновление), refetch при focus / reconnect / интервале, явная инвалидация после mutation, пометить «выдыхающимися» после staleTime. react-query это делает из коробки.

КАК РАБОТАЕТ

- 01 staleTime: время, в течение которого данные считаются свежими. После – stale.
- 02 Stale-данные показываются мгновенно из кэша + запускается фоновый refetch.
- 03 Triggers refetch: window focus, reconnect, mount, manual invalidate, interval.
- 04 cacheTime / gcTime: сколько кэш живёт после того, как никто на него не подписан.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой server-state.
- 02 Дашборды с регулярным обновлением.
- 03 Чат/notifications (часто focus-refetch).
- 04 Списки, где данные могут меняться.

ПОДВОДНЫЕ КАМНИ

- 01 staleTime=0 → запрос на каждый рендер (default).
- 02 staleTime=Infinity → данные никогда не обновляются.
- 03 Слишком частый refetch → нагрузка на сервер.
- 04 Без invalidate после mutation – UI показывает старое.

КОД

// Конфиг staleTime

```
useQuery({
  queryKey: ['user', id],
  queryFn: () => api.getUser(id),
  staleTime: 60_000,           // свежие 1 минуту
  refetchOnWindowFocus: true,
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем staleTime отличается от cacheTime?
 - 02 Как убрать refetchOnWindowFocus?
 - 03 Что такое stale-while-revalidate?
-

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/important-defaults>

Как инвалидировать cache?

ОПИСАНИЕ

Invalidation – пометить кэш «устаревшим» так, чтобы react-query или ваш store перезапросил данные. В react-query – `invalidateQueries({ queryKey })` после `mutation`. Гранулярность: `invalidate` целой ветки (`['users']`) или конкретного ключа (`['user', 1]`). В Redux – диспатчить `action` и заново тянуть данные.

КАК РАБОТАЕТ

- 01 `queryClient.invalidateQueries({ queryKey: ['user'] })` – inval всех queries, начинающихся с 'user'.
- 02 `exact: true` – только точное совпадение ключа.
- 03 После inval react-query автоматически `refetch active queries`.
- 04 `setQueryData` – обновить вручную (для optimistic).
- 05 `removeQueries` – полностью удалить из кэша.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 После любой `mutation`, меняющей серверные данные.
- 02 При событии «сменился пользователь / роль» – `invalidate` всё, что от него зависит.
- 03 При real-time-обновлениях через WebSocket.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком широкая инвалидация → много лишних запросов.
- 02 Слишком узкая → не обновляется связанное.
- 03 Иногда лучше `setQueryData` с патчем, чем `refetch`.

КОД

// После mutation

```
const m = useMutation({
  mutationFn: api.updateUser,
  onSuccess: () => {
    qc.invalidateQueries({ queryKey: ['user'] });
    qc.invalidateQueries({ queryKey: ['profile'] });
  },
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `invalidateQueries` отличается от `refetchQueries`?
- 02 Когда лучше `setQueryData`, чем `invalidate`?
- 03 Как разгружать инвалидацию связанных entities?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/query-invalidation>

NORMALIZATION**DERIVED****DUPLICATED****ASYNC FSM****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Архитектура хранилища: нормализация и производные данные.

ВОПРОСЫ ДНЯ

- Q23 Как проектировать normalized store?
- Q24 Когда normalization нужна?
- Q25 Когда normalization усложняет жизнь?
- Q26 Как работать с derived state?
- Q27 Как избежать duplicated state?
- Q28 Как проектировать async state machine?

ВОПРОС Q23

Как проектировать normalized store?

ОПИСАНИЕ

Normalized store хранит сущности по id в плоских коллекциях, а связи — через id-ссылки. Это убирает дублирование и упрощает обновления: «обновили юзера в одном месте — изменилось во всех экранах». Эталоны: { users: { byId: { '1': User }, allIds: ['1', ...] } }.

КАК РАБОТАЕТ

- 01 byId: { [id]: entity } — словарь.
- 02 allIds: string[] — порядок.
- 03 Связи — id или id[]: post.authorId, post.tagIds.
- 04 Селекторы достают полную entity по id.
- 05 RTK предоставляет createEntityAdapter с готовыми методами addOne/updateOne/removeOne.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие коллекции, которые показываются в разных местах.
- 02 Сущности с многосторонними связями (граф данных).
- 03 Когда дублирование стало источником багов.
- 04 Когда часть данных нужно обновлять отдельно от целого.

ПОДВОДНЫЕ КАМНИ

- 01 Излишняя нормализация для маленьких списков — больше boilerplate, чем выигрыша.
- 02 Удобство списков теряется — каждый раз нужны селекторы + join по id.
- 03 Сериализация в JSON — нет проблемы, но deep-equal становится сложнее.

КОД

// Схема

```
type State = {
  users: { byId: Record<string, User>; allIds: string[] };
  posts: { byId: Record<string, Post>; allIds: string[] };
};
// Post.authorId → User.id
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем createEntityAdapter в RTK помогает?
- 02 Когда нормализация — overkill?
- 03 Как с denormalization в react-query?

ПОЛНАЯ СТАТЬЯ

<https://redux.js.org/usage/structuring-reducers/normalizing-state-shape>

ВОПРОС Q24

Когда normalization нужна?

ОПИСАНИЕ

Когда сущность встречается во многих местах UI и приходит из API в разной обёртке (списки, детали, связанные). Без нормализации обновление одного юзера требует ручного прохода по всем местам, где он хранится. Особенно важно для real-time-данных и коллабораций.

КАК РАБОТАЕТ

- 01 Один источник правды на каждую сущность.
- 02 Селекторы строят views поверх нормализованной формы.
- 03 API-ответы денормализуют при загрузке.
- 04 Mutations обновляют только в одном месте.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Социальные сети, форумы – пользователь в комментариях, лайках, постах.
- 02 Real-time доски (Trello, Jira).
- 03 Большие админ-панели с CRUD.
- 04 Графовые данные (связи между сущностями).

ПОДВОДНЫЕ КАМНИ

- 01 Денормализация при загрузке – тяжёлая операция.
- 02 Без типизации легко поломать структуру.
- 03 Если API уже возвращает нормализованные данные – проще.

КОД

// Денормализация ответа

```
function normalizePosts(posts: ApiPost[]) {
  const users: Record<string, User> = {};
  const byId: Record<string, Post> = {};
  posts.forEach(p => {
    users[p.author.id] = p.author;
    byId[p.id] = { ...p, authorId: p.author.id };
  });
  return { users, byId };
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда лучше использовать GraphQL фрагменты вместо ручной нормализации?
- 02 Как с relay-style cursor pagination и нормализацией?

ПОЛНАЯ СТАТЬЯ

<https://redux.js.org/usage/structuring-reducers/normalizing-state-shape>

ВОПРОС Q25

Когда normalization усложняет жизнь?

ОПИСАНИЕ

Если данные простые: один список, без связей, без частых изменений – нормализация добавляет boilerplate без выгоды. Также для server-state, который ведёт react-query: его кэш и так структурирован, дополнительная нормализация дублирует работу. И в малых проектах с несколькими экранами – overkill.

КАК РАБОТАЕТ

- 01 Лишний слой денормализации/селекторов.
- 02 Сложность тестов – нужно проверять оба представления.
- 03 Trade-off: гибкость обновления vs простота разработки.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Маленькие приложения – нормализация лишняя.
- 02 Когда данные «короткоживущие» (одноразовый список).
- 03 Когда есть react-query / GraphQL клиент с нормализацией.

ПОДВОДНЫЕ КАМНИ

- 01 Преждевременная оптимизация структуры данных.
- 02 Слишком много слоёв абстракции.
- 03 Время на boilerplate vs реальная польза.

КОД

```
// Когда не надо
// маленький список из API
const { data: users } = useQuery({
  queryKey: ['users'],
  queryFn: api.getUsers,
});
// нормализация лишняя – react-query кэширует сам
```

МОГУТ ДОСПРОСИТЬ

- 01 Какой признак «здесь нужна нормализация»?
- 02 Как использовать react-query без нормализации?
- 03 Что делать, если данные становятся сложнее?

ПОЛНАЯ СТАТЬЯ

<https://redux.js.org/usage/structuring-reducers/normalizing-state-shape>

Как работать с derived state?

ОПИСАНИЕ

Derived state – вычисленные на лету значения из основного state. Не хранить отдельно, а вычислять при каждом render через простую функцию или мемоизированный селектор. Это убирает дубликаты и risks рассинхронизации. Дорогие – useMemo, mobx computed, reselect.

КАК РАБОТАЕТ

- 01 В компоненте: `const total = items.reduce(...)`.
- 02 Дорогая операция: `useMemo(() => calc(items), [items])`.
- 03 В Redux: `createSelector` из `reselect` – мемоизированные селекторы.
- 04 В MobX: `computed` автоматически кэшируется.
- 05 Анти-паттерн – хранить derived в state, синхронизировать вручную.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Считать `total/sum/filter` из массива.
- 02 Подсчёты на основе объединения нескольких state.
- 03 View-models поверх raw data.
- 04 Селекторы в Redux/Zustand.

ПОДВОДНЫЕ КАМНИ

- 01 Хранение derived в state → рассинхрон при обновлении основного.
- 02 Дорогие функции без `useMemo` – лишние вычисления.
- 03 `useMemo` с неправильными deps – кэш не работает.
- 04 Слишком много мелких `useMemo` – overhead больше выгоды.

КОД

// Простой derived

```
function Cart({ items }: { items: Item[] }) {
  const total = items.reduce((a, x) => a + x.price, 0);
  return <div>Total: {total}</div>;
}
```

// С мемо

```
const total = useMemo(
  () => items.reduce((a, x) => a + x.price, 0),
  [items]
);
```

// reselect

```
const selectTotal = createSelector(
```

```
[(s: State) => s.cart.items],  
  (items) => items.reduce((a, x) => a + x.price, 0),  
);
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда `derived` стоит мемоизировать?
- 02 Чем `reselect` отличается от `useMemo`?
- 03 Что такое "single source of truth" в этом контексте?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/you-might-not-need-an-effect#updating-state-based-on-props-or-state>

Как избежать duplicated state?

ОПИСАНИЕ

Duplicated state – одно и то же значение, хранящееся в нескольких местах (props + local copy, server + redux + context). Источник багов: одно обновляется, другое – нет. Принцип «single source of truth»: каждое значение имеет один владельца, остальные читают из него.

КАК РАБОТАЕТ

- 01 Один владелец – один setState/dispatch.
- 02 Другие компоненты читают через props/context/store.
- 03 Если нужно derived – вычислять, не хранить.
- 04 Если нужна локальная копия (например, draft перед submit) – она обязательно временная и явно отделена.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review – чек на дубли.
- 02 При проектировании state – задача поставить правильного владельца.
- 03 Для server-data использовать react-query, не копировать в Redux.

ПОДВОДНЫЕ КАМНИ

- 01 props → useState(props) – копия, которая не обновится при смене prop.
- 02 Sync через useEffect → infinite loop / выпадает из consistency.
- 03 Кэш в двух местах (Redux + react-query) – расхождение.

КОД

// Антипаттерн – копия props в state

```
function Bad({ user }: { user: User }) {  
  const [u, setU] = useState(user); // копия!  
  // u не обновится при смене user  
}
```

// Правильно – derived

```
function Good({ user }: { user: User }) {  
  return <div>{user.name}</div>;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое single source of truth?
- 02 Когда копия в state оправдана?
- 03 Как Reset state при смене props?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/sharing-state-between-components>

Как проектировать async state machine?

ОПИСАНИЕ

Async state – почти всегда конечный автомат: `idle → loading → success | error`. Лучшая модель – discriminated union с эксклюзивными вариантами. Это убирает невозможные состояния (`loading=true` и `data=...`). Можно реализовать через `useReducer`, `useState` с discriminated union, или `XState` (для сложных flow).

КАК РАБОТАЕТ

- 01 DU состояний: `{ kind: 'idle' } | { kind: 'loading' } | { kind: 'success'; data } | { kind: 'error'; error }`.
- 02 Экшены: `{ type: 'fetch' } | { type: 'success', data } | { type: 'fail', error }`.
- 03 `transition(state, action) → next state`.
- 04 Exhaustive switch для гарантии полноты.
- 05 `react-query` / `SWR` закрывает основные сценарии автоматически.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая async-загрузка с явными состояниями.
- 02 Multi-step операции (`upload`, `payment`).
- 03 Полностью повторяемые pipeline (`retry`, `cancel`).

ПОДВОДНЫЕ КАМНИ

- 01 Плоский `{ loading; data; error }` – невозможные состояния.
- 02 Без exhaustive – пропуск нового кейса.
- 03 Слишком много состояний – упускают связи.

КОД

// useReducer + DU

```
type State = { kind: 'idle' } | { kind: 'loading' }
  | { kind: 'success'; data: User } | { kind: 'error'; error: Error };
type Action = { type: 'fetch' } | { type: 'ok'; data: User }
  | { type: 'fail'; error: Error };

function reducer(s: State, a: Action): State {
  switch (a.type) {
    case 'fetch': return { kind: 'loading' };
    case 'ok':    return { kind: 'success', data: a.data };
    case 'fail':  return { kind: 'error', error: a.error };
  }
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем XState лучше useReducer для сложных flow?
- 02 Как обрабатывать cancel?
- 03 Что такое impossible states?

ПОЛНАЯ СТАТЬЯ

<https://statefy.ai/docs>

PERMISSIONS**FLAGS****REAL-TIME****PERF****TESTS****MISC****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Закрепление и операционные вопросы хранилища.

ВОПРОСЫ ДНЯ

- Q29 Как хранить permissions?
- Q30 Как хранить feature flags?
- Q31 Как проектировать real-time state?
- Q32 Как избежать re-render storm при частых updates?
- Q33 Как подписывать компоненты только на нужный slice?
- Q34 Как тестировать store?
- Q35 Как мигрировать state management без big bang rewrite?

ВОПРОС Q29

Как хранить permissions?

ОПИСАНИЕ

Permissions обычно живут в auth-state: множество ролей/прав, привязанных к текущему пользователю. Хранятся в Context (если редко меняются) или в store (если меняются динамически). Frontend проверяет UI, backend – реальную авторизацию (без надежды на frontend). Обычно загружаются один раз при login и обновляются при изменении сессии.

КАК РАБОТАЕТ

- 01 auth-store: { user, roles, permissions: Set<string> }.
- 02 useCan(action: string): boolean – хук-хелпер.
- 03 Контекст вокруг защищённых маршрутов.
- 04 Кэш на стороне клиента + invalidate при logout.
- 05 RBAC (роли) или ABAC (атрибуты).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая auth-страница / админ-панель.
- 02 Multi-tenant SaaS.
- 03 Любой UI с разной видимостью элементов.

ПОДВОДНЫЕ КАМНИ

- 01 Frontend-only check – не безопасность, только UX.
- 02 Дубль permissions в нескольких местах.
- 03 Кэш не инвалидируется при revoke – пользователь видит то, что нельзя.

КОД

```
// Хук useCan

function useCan(action: string): boolean {
  const perms = useAuth((s) => s.permissions);
  return perms.has(action);
}

// {useCan('user.delete') && <DeleteBtn />}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем RBAC отличается от ABAC?
- 02 Как обновлять permissions без relogin?
- 03 Что должен делать backend, чего не должен frontend?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/managing-state>

Как хранить feature flags?

ОПИСАНИЕ

Feature flags – конфиг, включающий/выключающий функциональность. Хранение: в Context (если редкий update) или store (для динамичных). Источник: API провайдера (LaunchDarkly, GrowthBook, Unleash) или build-time-конфиг. Кэшируются на старте, обновляются по необходимости.

КАК РАБОТАЕТ

- 01 useFlag(key: string): boolean – хук.
- 02 FlagsProvider грузит флаги при монтировании.
- 03 В TS – union literal-типов ключей.
- 04 Server-side: SSR с прокинутыми флагами.
- 05 Reset при смене user.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 A/B-эксперименты.
- 02 Gradual rollout.
- 03 Kill switch.
- 04 Скрытие фич за role/plan.

ПОДВОДНЫЕ КАМНИ

- 01 String-ключи без union – опечатки.
- 02 Forgotten flag – мусор в коде.
- 03 Гонки: флаг приходит асинхронно, до этого default.
- 04 FE/BE flag не синхронизированы.

КОД

// Базовое

```
type FlagKey = 'newDashboard' | 'paymentV2';
function useFlag(k: FlagKey): boolean {
  return useFlags((s) => s[k] ?? false);
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Как тестировать с разными значениями флагов?
 - 02 Где хранить default-значения?
 - 03 Чем GrowthBook лучше runtime if (env === ...)?
-

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#literal-types>

Как проектировать real-time state?

ОПИСАНИЕ

Real-time state синхронизируется с сервером по push-каналу (WebSocket, Server-Sent Events, push). Локальный кэш обновляется на каждое сообщение. Архитектура: subscriber → store update → UI. Ключевые вопросы – идемпотентность сообщений, упорядоченность, переподключение, инвалидация при reconnect.

КАК РАБОТАЕТ

- 01 WebSocket / SSE / Centrifugo / Pusher – channel.
- 02 При сообщении – update store или setQueryData в react-query.
- 03 Инвалидация при reconnect – refetch с актуальной версией.
- 04 Очередь сообщений: вне порядка → сортировка по timestamp/version.
- 05 DLQ для невалидных или duplicate.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Чаты, комментарии, live-комментарии.
- 02 Дашборды с обновлением метрик.
- 03 Realtime-коллаборации (Figma, Notion).
- 04 Уведомления / push.

ПОДВОДНЫЕ КАМНИ

- 01 Не идемпотентные обработчики → дубли.
- 02 Order delivery не гарантирован – нужны version numbers.
- 03 Reconnect без resync → stale данные.
- 04 Тяжёлый state на каждое сообщение → re-render storm.

КОД

// WS → setQueryData

```
ws.onmessage = (e) => {
  const msg = JSON.parse(e.data);
  if (msg.type === 'user.update') {
    qc.setQueryData(['user', msg.id], msg.user);
  }
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Как обрабатывать reconnect?
- 02 Что такое eventual consistency?
- 03 Чем CRDT помогает в коллаборации?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/updates-from-mutation-responses>

Как избежать re-render storm при частых updates?

ОПИСАНИЕ

Стратегии: батчить обновления (накапливать N мс, потом применять одним `setState`), use selectors с shallow-сравнением, throttling/debouncing, отдельный store для high-frequency данных, рендерить через canvas/requestAnimationFrame для анимированных значений.

КАК РАБОТАЕТ

- 01 Batching на стороне store: накопить 10-50 мс, потом merge.
- 02 Селекторы с мемо: `useStore(s => s.x, shallow)`.
- 03 throttle/debounce обновления подписки.
- 04 RAF для frame-rate-aware-обновлений.
- 05 Canvas / WebGL вместо DOM для тысяч динамических элементов.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Биржевые котировки.
- 02 WebSocket-чат с быстрыми сообщениями.
- 03 Realtime-карты, GPS.
- 04 Drag-drop с per-pixel updates.

ПОДВОДНЫЕ КАМНИ

- 01 Большой `setState` на каждое сообщение – kill UI.
- 02 Без shallow – селектор объекта вызывает ререндер на каждом.
- 03 Canvas теряет accessibility – нужны fallback'и.
- 04 Тестирование становится сложнее.

КОД

// Batching через интервал

```
let pending: Update[] = [];  
function onMessage(u: Update) {  
  pending.push(u);  
  if (pending.length === 1) setTimeout(flush, 50);  
}  
function flush() {  
  store.applyBatch(pending);  
  pending = [];  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда переходить на canvas?
- 02 Чем shallow в Zustand отличается от deep?
- 03 Как тестировать throttle-логику?

ПОЛНАЯ СТАТЬЯ

<https://docs.pmnd.rs/zustand/guides/typescript>

Как подписывать компоненты только на нужный slice?

ОПИСАНИЕ

Селекторы. В Zustand – `useStore(s => s.x)`; в Redux – `useSelector(s => s.x)`; в MobX – `observer` + ссылка только на нужное поле; в Context – `use-context-selector`. Принцип: подписка читает минимально необходимое значение, сравнивается через `Object.is`, ререндер только при изменении именно этого slice.

КАК РАБОТАЕТ

- 01 Селектор – функция `(state) => slice`.
- 02 Сравнение по умолчанию `Object.is` (ссылочное).
- 03 `shallow` / `deep` сравнение – для объектов/массивов.
- 04 `createSelector` (`reselect`) – мемоизация.
- 05 `use-context-selector` – workaround для Context.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Везде, где есть `store` / `Context`.
- 02 В оптимизации перфоманса дашбордов.
- 03 В большом state с многими подписчиками.

ПОДВОДНЫЕ КАМНИ

- 01 Селектор объекта без `shallow` – ререндер на каждое изменение.
- 02 Inline-селекторы `create-on-render` – TS может поджаловаться.
- 03 Слишком гранулярные подписки – сложно отследить.

КОД

// Zustand

```
const x = useStore((s) => s.x);           // primitive
const xy = useStore((s) => ({ x: s.x, y: s.y }), shallow);
// shallow из 'zustand/shallow' для объекта
```

// reselect

```
const selectTotal = createSelector(
  [(s: RootState) => s.cart.items],
  (items) => items.reduce((a, x) => a + x.price, 0)
);
const total = useSelector(selectTotal);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `shallow` и `deep` сравнение?
- 02 Чем `reselect` помогает в Redux?

03 Когда лучше выделить отдельный store?

ПОЛНАЯ СТАТЬЯ

<https://docs.pmnd.rs/zustand/guides/typescript#bounded-usestore-hook-for-vanilla-stores>

Как тестировать store?

ОПИСАНИЕ

Store без UI можно тестировать как обычные функции: вызвать action → проверить snapshot. Сценарии: начальное state, переходы, async-thunks (с mock API), edge cases. Для UI-интеграции – testing-library с провайдером store. Mock API – MSW (mock service worker) или fetch-mock.

КАК РАБОТАЕТ

- 01 Тесты store: импорт, dispatch action, проверка getState.
- 02 Async: mock API → action → проверка состояния после.
- 03 UI: render с провайдером, fire event, проверка через screen.
- 04 Snapshot-тесты для derived selectors.
- 05 MSW – для realistic mock без modify fetch.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой нетривиальный store.
- 02 Reducers и selectors всегда тестируемы.
- 03 Async-thunks / mutations – тоже unit.
- 04 UI-интеграция – отдельный слой.

ПОДВОДНЫЕ КАМНИ

- 01 Тесты с реальным API → flaky.
- 02 Слишком гранулярные тесты – высокая поддержка.
- 03 Без mock-фабрик – boilerplate.
- 04 Side effects (analytics, navigation) забыли замаскировать.

КОД

```
// Test Zustand-store
test('inc', () => {
  const { result } = renderHook(() => useStore());
  act(() => result.current.inc());
  expect(result.current.count).toBe(1);
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое MSW и зачем?
 - 02 Чем Vitest отличается от Jest?
 - 03 Как тестировать middleware?
-

ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/react-testing-library/intro/>

Как мигрировать state management без big bang rewrite?

ОПИСАНИЕ

Strangler-pattern: новые фичи пишутся на новом store, старые – постепенно переезжают. Можно держать оба store одновременно, синхронизировать через адаптер. Шаги: identify boundaries → выбрать pilot-фичу → перенести → проверить → расширять. На больших проектах – месяцы работы по чуть-чуть.

КАК РАБОТАЕТ

- 01 Параллельная инсталляция Zustand рядом с Redux.
- 02 Адаптер: Redux-state → Zustand-store при изменении (или наоборот).
- 03 Pilot – одна фичка, желательно изолированная.
- 04 Постепенно убирать старые reducers.
- 05 Метрика – % кода на новом store, количество дубликатов.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Старый Redux → react-query + Zustand.
- 02 MobX → Redux Toolkit.
- 03 Custom event-bus → useSyncExternalStore.
- 04 Любая большая миграция в проде.

ПОДВОДНЫЕ КАМНИ

- 01 Big-bang rewrite – обычно убивает проект.
- 02 Без plan'a – миграция растягивается на годы.
- 03 Adapter с дублированием – источник багов.
- 04 Команда тратит время на миграцию вместо фич.

КОД

// Адаптер двух store

```
// Redux-action синхронит Zustand
store.subscribe(() => {
  const newAuth = store.getState().auth;
  useAuthZ.setState({ user: newAuth.user });
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое strangler-pattern?
 - 02 Как мерить прогресс миграции?
 - 03 Когда лучше переписать сразу?
-

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/StranglerFigApplication.html>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- [] Сказать, когда хватает local, когда нужен Context, и какие у Context риски (Q1, Q2, Q3, Q4, Q5)

- [] Выбрать инструмент: Redux / Zustand / MobX / XState – по сценарию (Q6, Q7, Q8, Q9, Q10)

- [] Развести client-state и server-state, описать роль react-query (Q11, Q12, Q13)

- [] Расписать boundaries: что в URL / Storage / memory, как синхронить tabs (Q14, Q15, Q16, Q17, Q18)

- [] Описать optimistic update + откат + cache invalidation (Q19, Q20, Q21, Q22)

- [] Объяснить normalization, derived state, как избегать дубликатов (Q23, Q24, Q25, Q26, Q27)

- [] Спроектировать async state machine (Q28)

- [] Сказать, как хранить permissions, feature flags, real-time state (Q29, Q30, Q31)

- [] Назвать 2 способа избежать re-render storm + slice-подписку (Q32, Q33)

- [] Описать тестирование store и стратегию миграции без big bang (Q34, Q35)
